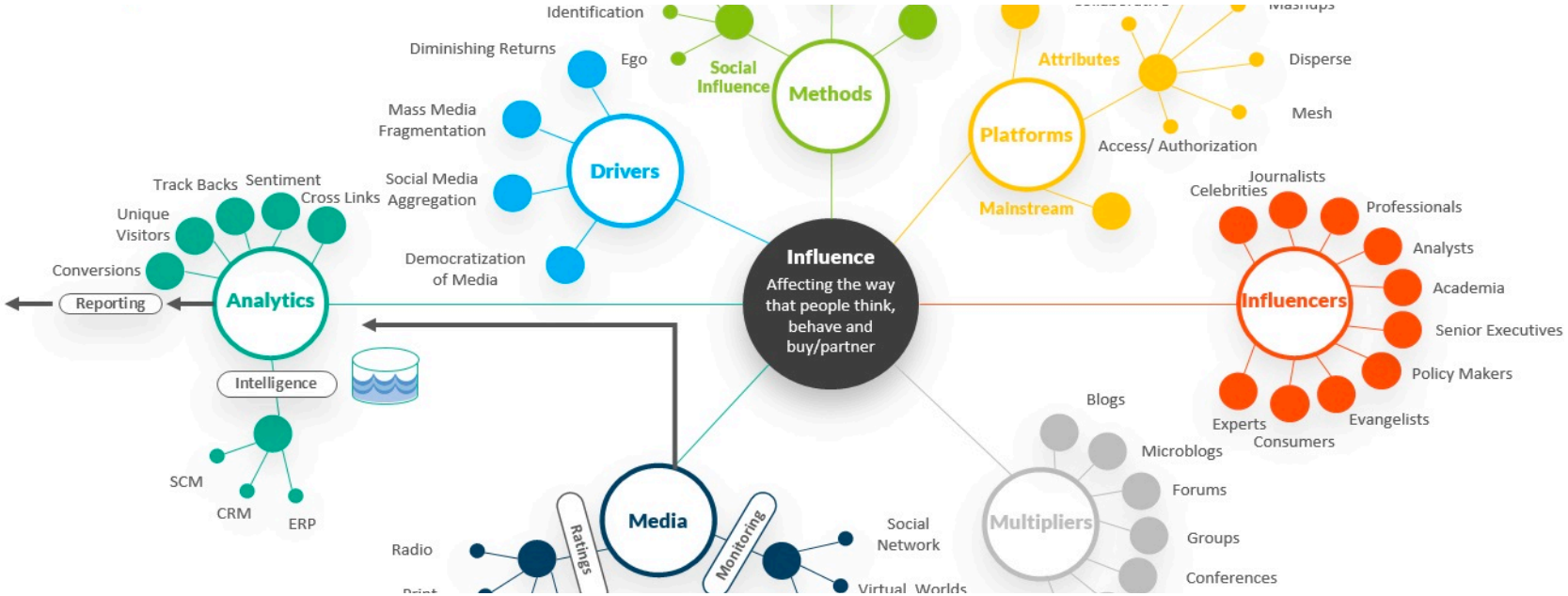
Term Project



Article Recommender Service

Agenda

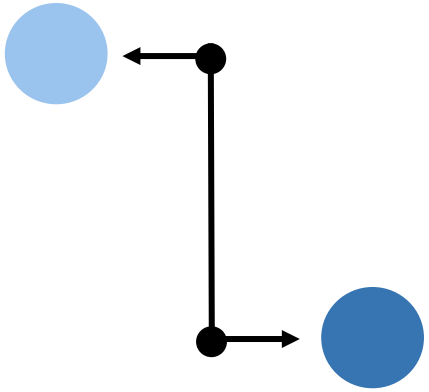
- Business Problem
- Solution Overview
- Implementation Approach
- LDA – Implementation Steps
- LDA – Define Vectors & Calculate Distance
- LDA – Topic Clusters
- Evaluation - LDA with Topic Coherence

Business Problem



Keyword Search - Articles

If you have a dataset of more than 100K plus articles and we would to filter specific articles which contain search query “keyword” then it will helpful to focus on filtered based returned article. Article should be filtered based on matching keyword in “Title” & “Text” of respective article



Question Search - Articles

If you have a dataset of more than 100K plus articles and we would to discover specific articles against a search question then it will helpful to focus on filtered articles relevant to the question. Article should be filtered based on matching context in “Title” & “Text” of respective article

Solution - Overview

Discover number of topic using Latent Dirichlet Allocation - LDA

Map specific article into the topic vector space

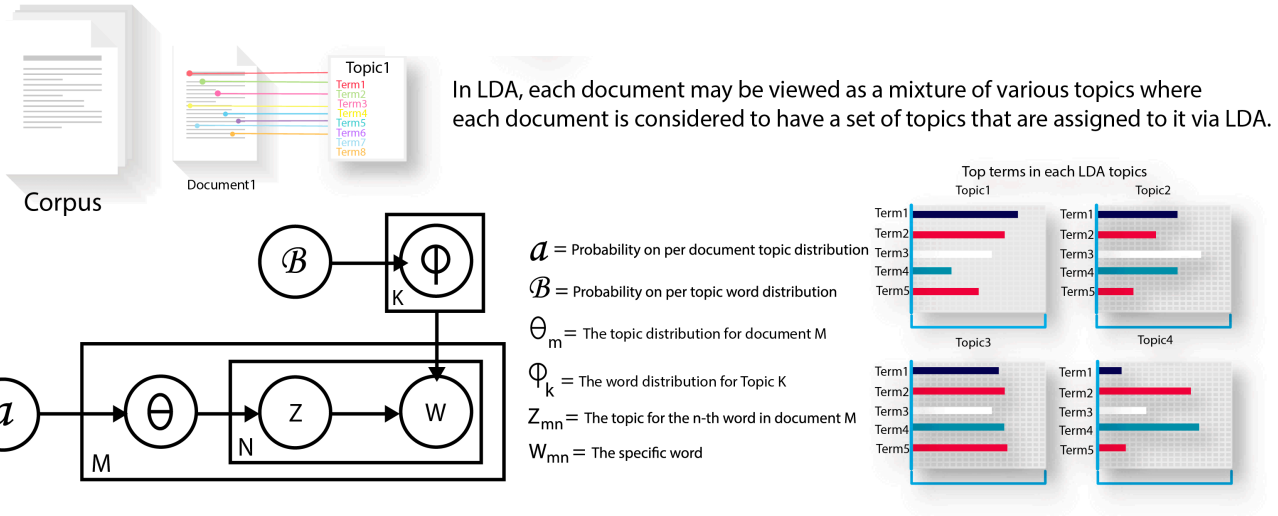
Use Jensen Shannon distance to find related articles

10000 Webhoise articles

20% Filtered as Deduplicate

7500+ Unique Articles

Recommend Top 5



Implementation Approach

Dataset

We are consuming WEBHOISEIO to pull 10000 article with following filter

Keyword in text = COVID or Corona

Publication = Last 1 month with crawler set to last 30 days

Site Category = News

Language - English

Deduplication

Remove duplicates using Simhash + Word2vec – This will help us to have unique article titles while keeping only one copy

Topic Modeling & Clustering

Idea to apply topic modelling to covid-19 related articles and define topic distribution then find clusters with 2-4 ngram to explore top keyword repetition. LDA (Latent Dirichlet Allocation) helps to classify text in document to a particular topic & word per topic to give us vector value of each topic.

Find Nearest Article in Topic Space

Jensen-Shanoon divergence method will be used to measure similarity of article in same topic space.

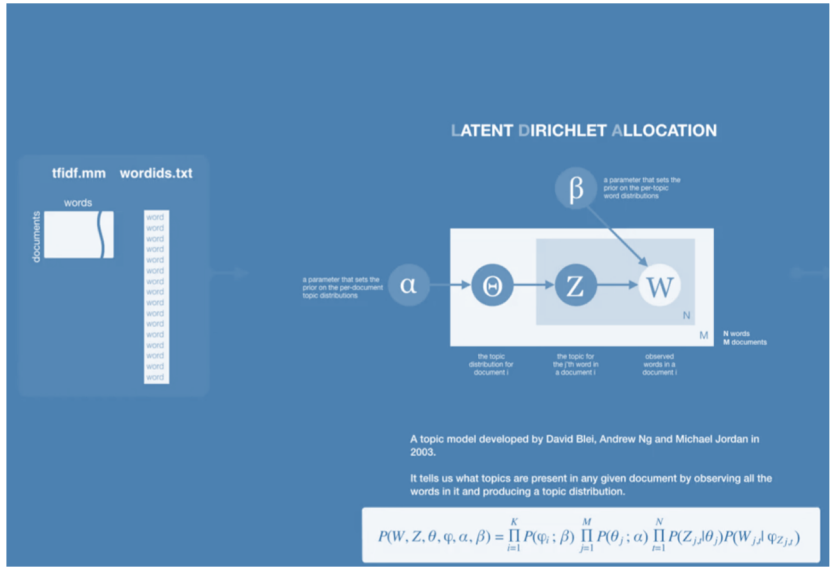
- topic distribution (sums to 1) of one article
- Returns the index of the k nearest articles (as by Jensen–Shannon divergence in topic space)

Backup options:
Semantic text similarity (Word2Vec)

MAP Article to Question + Find Relevant Article

TOP FAQs are mapped to relevant articles and then it can show top 5 relevant articles, similarly we can pick any article and it shows similar content based article as a choice

LDA – Implementation Steps





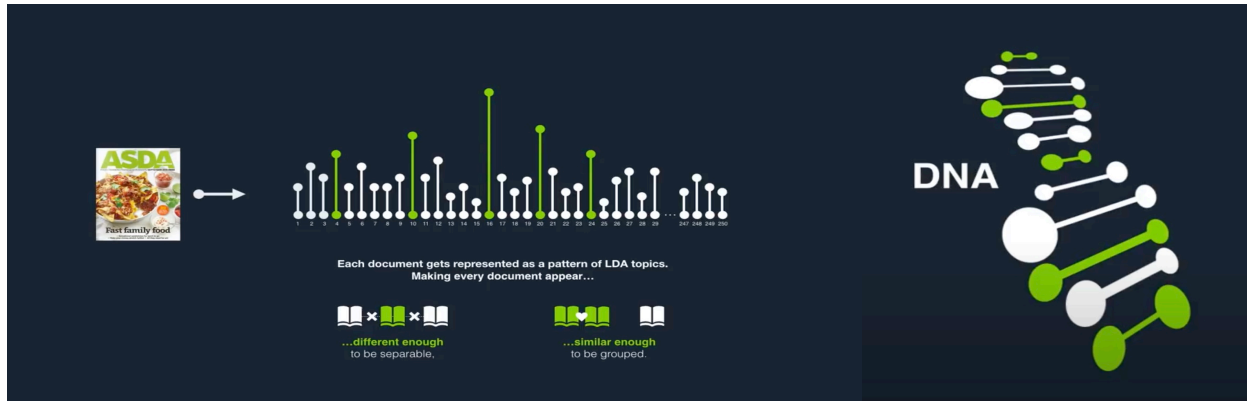
In [40]:
```
doc_topic_dist = pd.DataFrame(lda.transform(data_vectorized))
doc_topic_dist.to_csv('doc_topic_dist.csv', index=False)
```

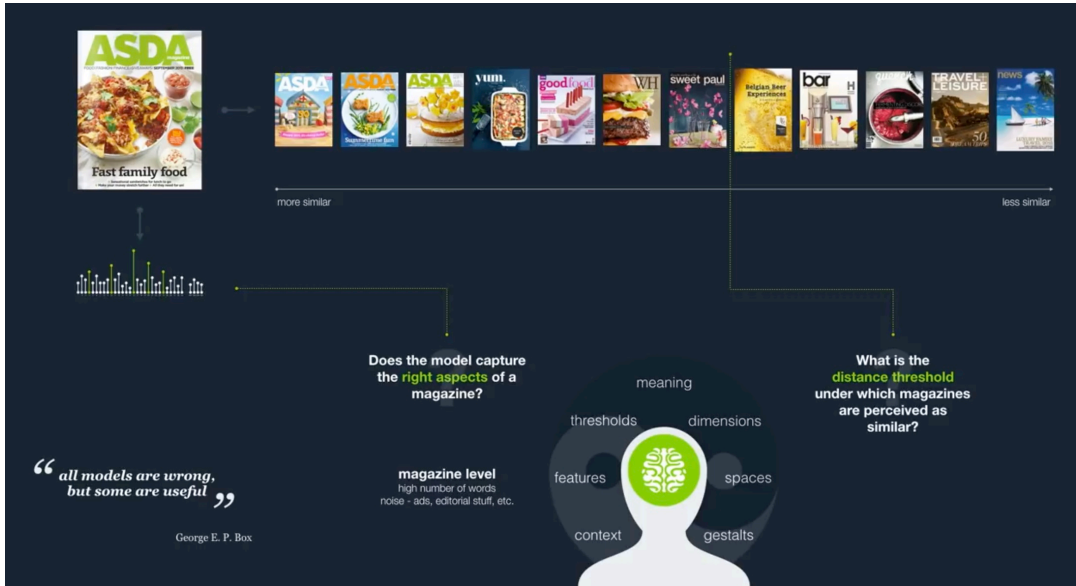
In [41]:
```
doc_topic_dist.head()
```

Out[41]:

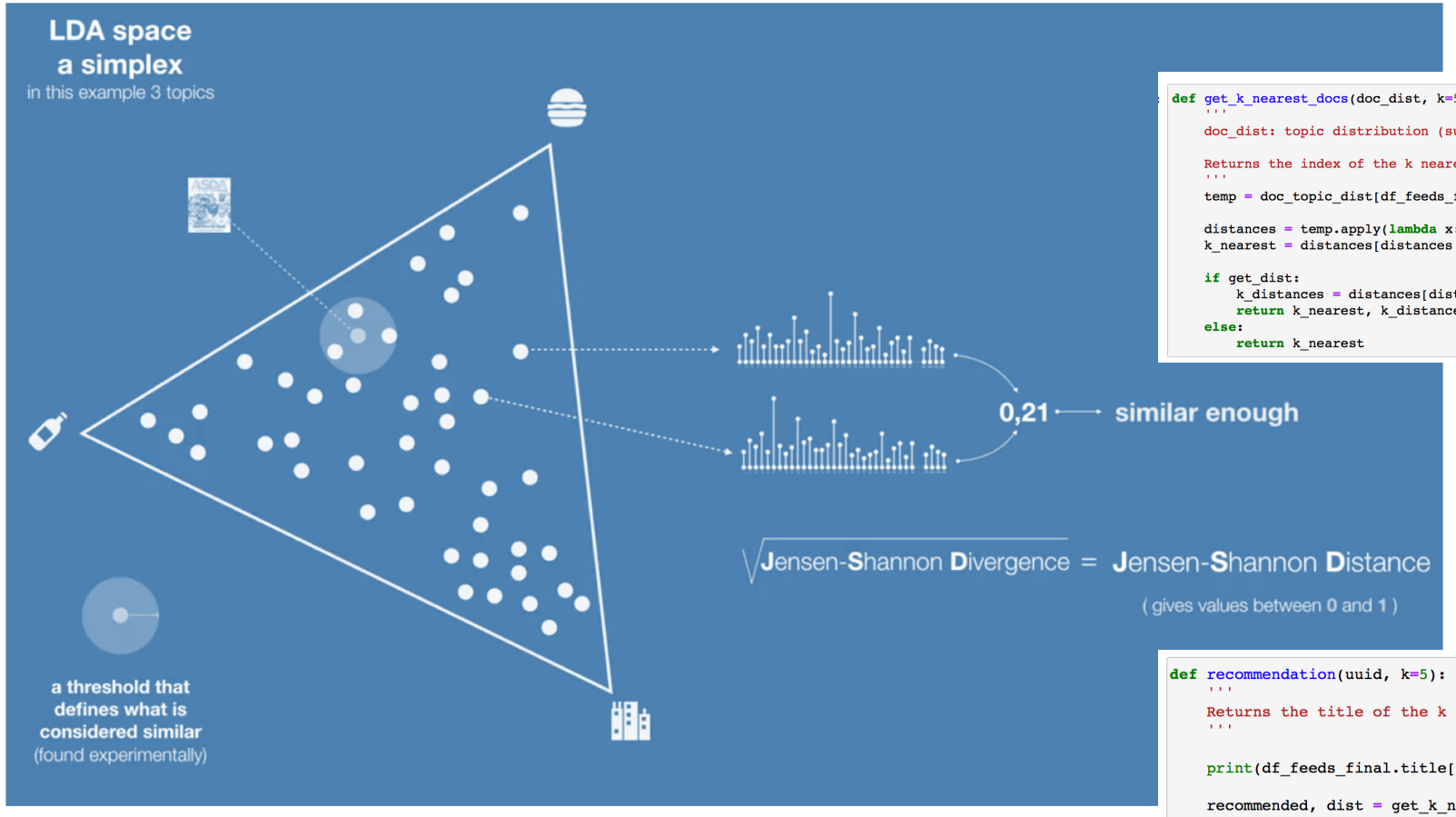
	0	1	2	3	4	5	6	7	8	9	...	40
0	0.010000	0.010000	0.010000	0.010000	0.010000	0.010000	0.010000	0.010000	0.010000	0.010000	...	0.010000
1	0.227607	0.000028	0.000028	0.000028	0.208188	0.022621	0.000028	0.000028	0.000028	0.000028	...	0.057881
2	0.000063	0.010302	0.000063	0.000063	0.029159	0.000063	0.255754	0.019734	0.000063	0.000063	...	0.000063
3	0.000152	0.000152	0.000152	0.000152	0.000152	0.000152	0.000152	0.000152	0.000152	0.000152	...	0.000152
4	0.000033	0.000033	0.000033	0.000033	0.060591	0.117044	0.000033	0.000033	0.000033	0.000033	...	0.000033

5 rows × 50 columns





LDA – Define Vector & Calculate Distance




```python
def get_k_nearest_docs(doc_dist, k=5, get_dist=False):
    '''
    doc_dist: topic distribution (sums to 1) of one article

    Returns the index of the k nearest articles (as by Jensen–Shannon divergence in topic space).
    '''
    temp = doc_topic_dist[df_feeds_final.published.between('2020-03-01T04:54:00.000+02:00', '2020-03-31T04:54:00.000+02:00')]

    distances = temp.apply(lambda x: jensenshannon(x, doc_dist), axis=1)
    k_nearest = distances[distances != 0].nsmallest(n=k).index

    if get_dist:
        k_distances = distances[distances != 0].nsmallest(n=k)
        return k_nearest, k_distances
    else:
        return k_nearest
```

```python
def recommendation(uuid, k=5):
    '''
    Returns the title of the k papers that are closest (topic-wise) to the paper given by paper_id.
    '''

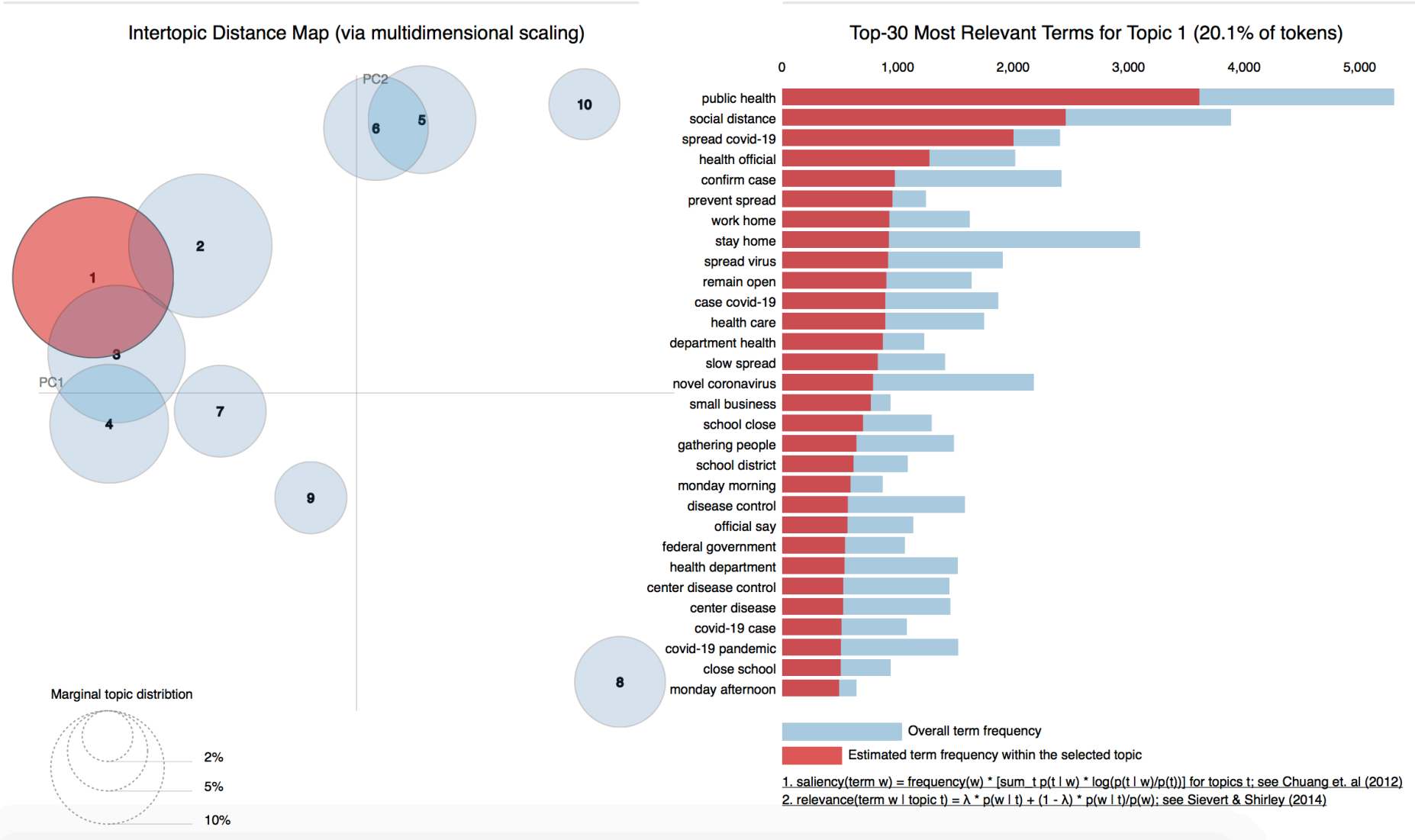
    print(df_feeds_final.title[df_feeds_final.uuid == uuid].values[0])

    recommended, dist = get_k_nearest_docs(doc_topic_dist[df_feeds_final.uuid == uuid].iloc[0], k, get_dist=True)
    recommended = df_feeds_final.iloc[recommended].copy()
    recommended['similarity'] = 1 - dist

    h = '<br/>'.join(['<a href="' + l + '" target="_blank">'+ n + '</a>' +' (Similarity: ' + "{:.2f}".format(s) + ')
    display(HTML(h))
```

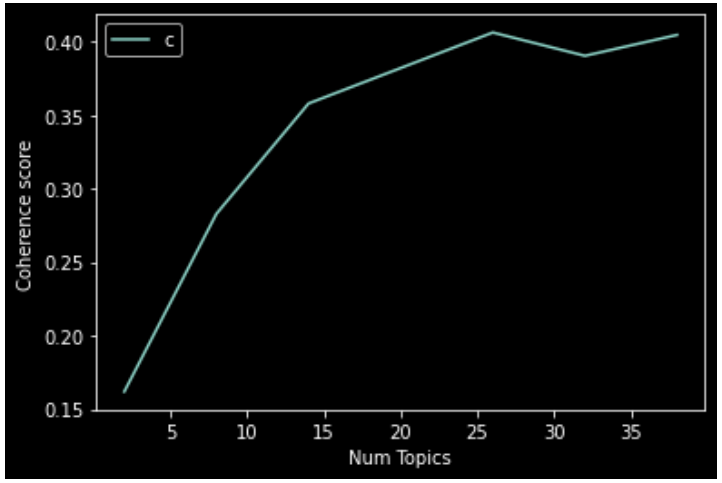
- Calculate Vector Value
- JSD to find Distance
- Alternate is Cosine Value
- Closed circle will be recommended

LDA – Topic Clusters



Evaluation – LDA with Topic Coherence

- **Eye Balling Models**
  - Top N words
  - Topics / Documents
- **Topic Coherence - Intrinsic Evaluation Metrics**
  - Capturing model semantics
  - Topics interpretability
- **Human Judgements**
  - What is a topic




```
d = get_k_nearest_docs(doc_topic_dist[df_feeds_final.uuid == '78bed2ff0e89a17197c4cc5a5394230fea71e167'].iloc[0])

sb.kdeplot(d)
```

```
#7d6f675469a5a58a0179034faad7c3880d23d0e7

recommendation('5bbc290d8417b748bfa7aa0bcd98e2e7c4c8e8e3', k=5)
```

```
'You cannot fight a fire blindfolded': WHO chief blasts slow virus testing response | UN News
```

'You cannot fight a fire blindfolded': WHO chief blasts slow virus testing response (Similarity: 0.92)
Commitment, Transparency Pay Off as South Korea Limits COVID-19 Spread (Similarity: 0.62)
BREAKING: Clear Reports Coronavirus Positive Patient Entered Zimbabwe And Govt And UN Lied. (Similarity: 0.62)
Number of coronavirus cases in Turkish metropolis "has exploded" (Similarity: 0.61)
Coronavirus outbreak: WHO says key to stopping COVID-19 spread is 'test and isolate' (Similarity: 0.60)

```
def compute_coherence_values(dictionary, corpus, texts, limit, start=2, step=3):
    """
    Compute c_v coherence for various number of topics

    Parameters:
    ----------
    dictionary : Gensim dictionary
    corpus : Gensim corpus
    texts : List of input texts
    limit : Max num of topics

    Returns:
    -------
    model_list : List of LDA topic models
    coherence_values : Coherence values corresponding to the LDA model with respective number of topics
    """
    coherence_values = []
    model_list = []
    for num_topics in range(start, limit, step):
        model=LdaModel(corpus=corpus, id2word=dictionary, num_topics=num_topics)
        model_list.append(model)
        coherencemodel = CoherenceModel(model=model, texts=texts, dictionary=dictionary, coherence='c_v')
        coherence_values.append(coherencemodel.get_coherence())

    return model_list, coherence_values
```